Module 8: Functions

What is a Function?

- A **function** is a reusable block of code that performs a specific task.
- Helps break a program into smaller, manageable pieces.
- Improves **readability**, **modularity**, and **reusability**.

Function Definition vs Function Use

- **Function definition**:
  - The actual implementation (what the function does).
- **Function use (call)**:
  - Invoking the function to execute its code.

```
// Definition
int add(int a, int b) {
    return a + b;
}

// Use (call)
int result = add(3, 4);
```

Function Syntax

PROF

General form:

```
return_type function_name(parameter_list) {
    // body
}
```

Example:

```
int square(int x) {
    return x * x;
}
```

Parameters and Return Type

- **Parameters**:
  - Inputs to the function.
  - Declared in the parameter list.
- **Return type**:
  - Type of value the function returns.
  - Use `void` if no value is returned.

Examples:

```
void greet() {
    printf("Hello!\n");
}

int multiply(int a, int b) {
    return a * b;
}
```

How to Call a Function

- Use the function name followed by parentheses.
- Provide required arguments.

```
int result = multiply(2, 5);
greet();
```

PROF

- Execution jumps to the function, then returns back.

Function Call as an Expression

- A function call can be used like a value.

```
int x = square(4) + 10;
printf("%d", square(3));
```

- The returned value replaces the function call.

Argument Passing (Pass by Value)

- In C, arguments are passed **by copying values**.
- The function works on **copies**, not original variables.

```c
void change(int x) {
    x = 10;
}

int main() {
    int a = 5;
    change(a);
    // a is still 5
}
```

- Original variables are unaffected.

Function Prototype vs Definition

- **Prototype (declaration)**:
    - Tells the compiler about the function before use.
    - No body.

```c
int add(int a, int b);  // prototype
```

- **Definition**:
    - Actual implementation.

```c
int add(int a, int b) {
    return a + b;
}
```

PROF

- Needed when function is used before it is defined.

The `return` Statement

- Ends function execution.
- Sends a value back to the caller.

```c
int max(int a, int b) {
    if (a > b)
        return a;
```

```
    return b;
}
```

- In `void` functions, `return;` is optional.

Summary

- Functions modularize code.
- Defined with return type, name, and parameters.
- Called using function name and arguments.
- Calls behave like expressions.
- Arguments are passed by value (copied).
- Prototypes declare functions before use.
- `return` sends results back to caller.

Practice Exercise

Write a function:

- Name: `is_even`
- Input: integer
- Output: 1 if even, 0 if odd

Example:

```
int is_even(int n) {
    return n % 2 == 0;
}
```
